



Instituto Superior de Engenharia de Lisboa  
Licenciatura em Engenharia Informática e Multimédia

## Fundamentos de Sistemas Operativos

Docente: Eng.º Carlos Carvalho

Semestre de Inverno 25/26

### **1º Trabalho Prático – Parte 4**

**Trabalho realizado por:**

Miguel Cordeiro 49765 LEIM32D Grupo 10

Ricardo Ferreira 51324 LEIM32D Grupo 10

Rodrigo Medina 51639 LEIM32D Grupo 10

Lisboa, 20 de Outubro de 2025

# Índice

|      |                                        |    |
|------|----------------------------------------|----|
| 1.   | Introdução .....                       | 3  |
| 2.   | Desenvolvimento .....                  | 4  |
| 3.   | Conclusões .....                       | 7  |
| 4.   | Bibliografia.....                      | 8  |
| 5.   | Anexo-Código .....                     | 9  |
| 5.1. | Classe “Application.java” .....        | 9  |
| 5.2. | Classe “BaseDados.java” .....          | 10 |
| 5.3. | Classe “GUI.java” .....                | 11 |
| 5.4. | Classe “Tarefa.java” .....             | 24 |
| 5.5. | Classe “Comando.java” .....            | 26 |
| 5.6. | Classe “BufferCircular.java” .....     | 27 |
| 5.7. | Classe “Servidor.java” .....           | 29 |
| 5.8. | Classe “MovimentoAleatorio.java” ..... | 31 |
| 5.9. | Classe “RobotLegoEV3SIM.java” .....    | 33 |

## Índice de figuras

|                                      |   |
|--------------------------------------|---|
| Figura 1 - Diagrama de classes. .... | 6 |
|--------------------------------------|---|

## 1. Introdução

No seguimento do trabalho desenvolvido na terceira aula prática, foi dada continuidade à implementação do projeto, tendo como base os diagramas estabelecidos.

Esta sessão teve como principais objetivos:

- Implementar a classe `MovimentoAleatorio`, baseada na estrutura abstrata `Tarefa`, permitindo gerar e inserir no buffer lotes de comandos de forma concorrente e controlada.
- Implementar o controlo dos Movimentos Aleatórios através de semáforos, assegurando exclusão mútua no acesso ao `BufferCircular` e evitando bloqueios.

## 2. Desenvolvimento

Nesta segunda aula prática foi dada continuidade ao projeto iniciado anteriormente, focando-se principalmente na parte do controlo automático do robot e na comunicação entre as várias tarefas do sistema. O objetivo foi permitir que o robot pudesse ser controlado tanto de forma manual (através da interface gráfica) como automática, usando comandos gerados aleatoriamente.

A classe `MovimentoAleatorio` foi criada como uma subclasse da classe `Tarefa`, funcionando como um produtor de comandos. Esta tarefa tem a responsabilidade de gerar movimentos aleatórios e colocá-los no buffer circular.

Durante a execução, são criados vários comandos seguidos, que simulam diferentes tipos de movimentos do robot (por exemplo: andar para a frente, virar à esquerda ou à direita). No fim de cada conjunto de comandos é sempre enviado um comando `PARAR`, que serve para indicar o final do lote.

Foi implementado um modo contínuo, que pode ser ativado na interface através de um botão de rádio. Quando este modo está ativo, a tarefa gera automaticamente lotes de comandos de dois em dois segundos, sem ser necessário o utilizador intervir.

A classe `Comando` foi desenvolvida para representar as várias instruções que o robot pode executar. Cada comando guarda o seu tipo (por exemplo, `RETA_FRENTE`, `CURVA_DIR`, `PARAR`, etc.) e os valores necessários, como distância, raio ou ângulo. Foram criadas funções estáticas que simplificam a criação dos comandos, como `Comando.retaFrente(int distancia)` ou `Comando.curvaDir(int raio, int angulo)`. Desta forma, o código da GUI e do servidor ficou mais limpo, uma vez que já não precisam de construir os comandos manualmente.

Esta estrutura facilita também a manutenção do projeto, pois se for necessário adicionar novos tipos de movimentos, basta criar um novo método dentro da classe `Comando`.

A interface gráfica foi modificada para deixar de controlar diretamente o robot e passar a comunicar através do buffer. Agora, cada botão da GUI (Frente, Trás, Esquerda, Direita e Parar) cria um objeto do tipo `Comando` e envia-o para o `BufferCircular`. O servidor, que funciona como consumidor, lê os comandos e executa as ações correspondentes no robot. Foi ainda adicionado o controlo dos movimentos aleatórios, que pode ser ligado ou desligado com o botão de rádio. Quando está ligado, os botões manuais são desativados

para evitar que o utilizador envie comandos enquanto o modo automático está a funcionar, garantindo que não há interferência entre os dois modos.

Para coordenar o acesso ao buffer entre os vários produtores e o consumidor, foram usados três semáforos:

O primeiro, “elementosLivres”, impede que os produtores escrevam no buffer quando este está cheio.

O segundo, “elementosOcupados”, faz o oposto: evita que o consumidor tente ler quando o buffer está vazio.

Por fim, o semáforo “acessoElemento”, garante exclusão mútua, ou seja, que só uma tarefa de cada vez pode mexer no buffer, impedindo acessos simultâneos que poderiam causar erros.

Graças a este controlo por semáforos, é possível garantir que os comandos vindos da GUI não se misturam com os comandos gerados automaticamente. Assim, o sistema mantém a ordem correta de execução e evita conflitos entre diferentes produtores.

Nesta secção, é também apresentado o novo diagrama de classes do projeto, elaborado após as alterações realizadas no código. O diagrama foi reconstruído no software MagicDraw, refletindo com maior precisão a estrutura e os relacionamentos entre as classes do sistema após a refatoração e implementação das novas funcionalidades.

O objetivo desta atualização foi garantir a coerência entre a implementação e o modelo UML, permitindo uma melhor compreensão da arquitetura do sistema e facilitando a manutenção futura.

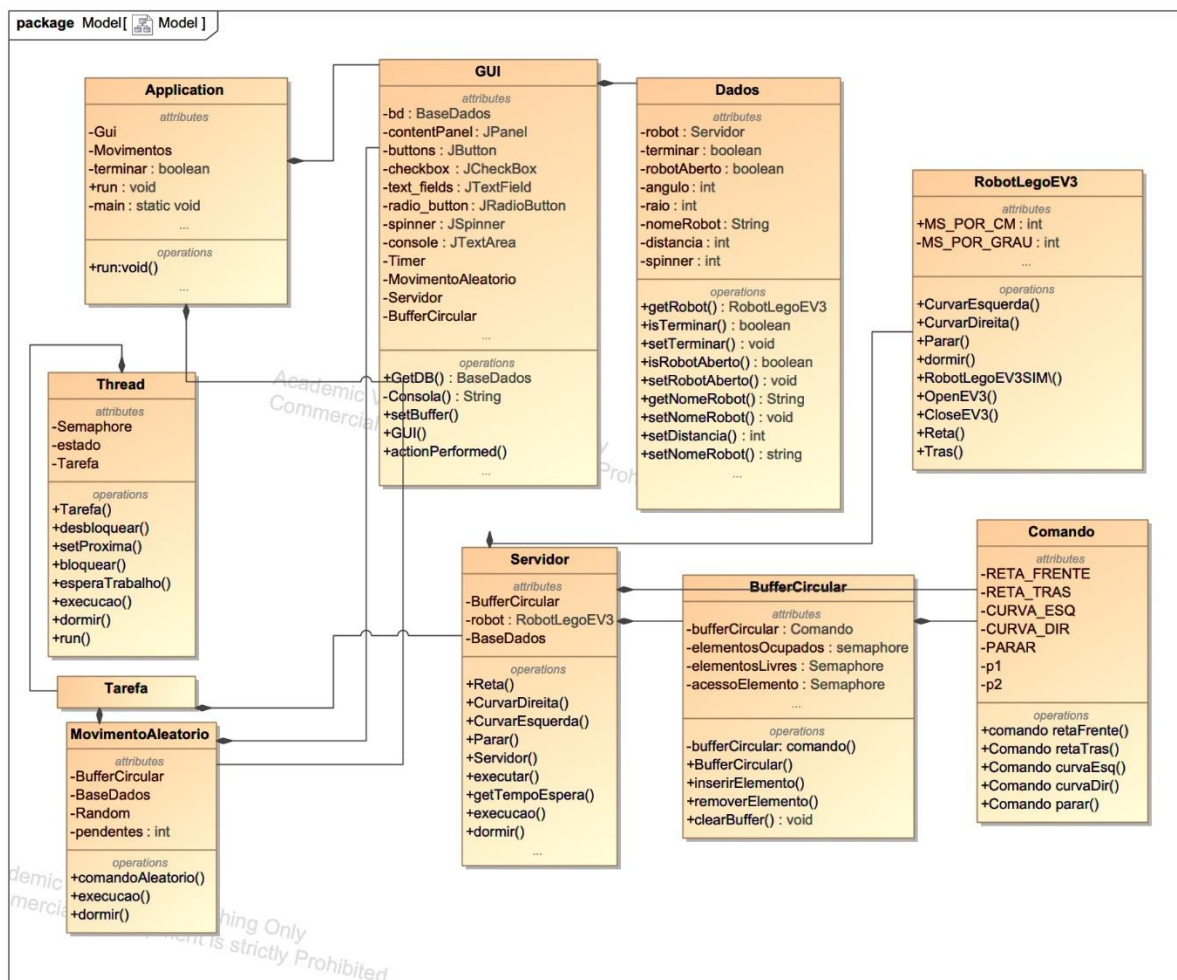


Figura 1 - Diagrama de classes.

### 3. Conclusões

Com esta segunda fase do projeto foi possível consolidar vários conceitos de programação concorrente e de comunicação entre tarefas. A implementação do buffer circular com semáforos revelou-se fundamental para garantir o controlo correto entre os produtores (a GUI e a tarefa dos movimentos aleatórios) e o consumidor (o servidor que envia os comandos ao robot).

A introdução da classe Comando simplificou bastante o código e tornou o sistema mais organizado, permitindo separar a criação dos comandos da sua execução. Por outro lado, a classe MovimentoAleatorio trouxe uma vertente mais autónoma ao projeto, permitindo ao robot realizar movimentos sem intervenção direta do utilizador.

A adaptação da interface gráfica para trabalhar com comandos e o controlo do modo automático mostraram na prática a importância da sincronização entre diferentes partes de um programa multithread. Verificou-se também que o uso correto dos semáforos impede que os comandos manuais interfiram com os lotes automáticos, assegurando o bom funcionamento do sistema.

De forma geral, esta etapa do trabalho permitiu compreender melhor como aplicar mecanismos de exclusão mútua, coordenação entre tarefas e estruturas de comunicação partilhadas em Java, aproximando o projeto de uma aplicação real de controlo concorrente de um robot.



## **4. Bibliografia**

Pais, Jorge. Fundamentos de Sistemas Operativos. 2025-2026.

Silberchatz A., Galvin P., Gagne G., Operating System Concepts with Java, John Wiley & Sons, 2009.

## 5. Anexo-Código

### 5.1. Classe “*Application.java*”

```
public class Application {  
    private GUI gui;  
    private MovimentoAleatorio produtor;  
    private Servidor servidor;  
    private BufferCircular buffer;  
    private BaseDados bd;  
  
    public Application() {  
        gui = new GUI();  
        bd = gui.getBD();  
  
        buffer = new BufferCircular();  
        produtor = new MovimentoAleatorio(buffer, bd);  
        servidor = new Servidor(buffer, bd.getRobot(), bd);  
        produtor.start();  
        servidor.start();  
  
        gui.setTarefas(produtor, servidor);  
        gui.setBuffer(buffer);  
    }  
  
    public void run() {  
  
        System.out.println("A aplicação começou.");  
        while (!gui.getBD().isTerminar()) {  
            try {  
                Thread.sleep(100);  
            } catch (InterruptedException e) {  
                // TODO Auto-generated catch block  
                e.printStackTrace();  
            }  
        }  
    }  
}
```

```
        System.out.println("A aplicação terminou.");
    }
    public static void main (String[] args) {
        Application app = new Application();
        app.run();
    }
}
```

## 5.2. Classe “BaseDados.java”

```
public class BaseDados {
    private boolean terminar;
    private boolean robotAberto;
    private RobotLegoEV3 robot;
    private int distancia;
    private int angulo;
    private int raio;
    private String nomeRobot = "EVA";

    public BaseDados() {
        terminar = false;
        robotAberto = false;
        //robot = new RobotLegoEV3();
        robot = new RobotLegoEV3SIM(mensagem -> System.out.println(mensagem));
        //distancia = 0;
        //angulo = 0;
        //raio = 0;
    }

    public RobotLegoEV3 getRobot() { return robot; }
    public boolean isTerminar() { return terminar; }
    public void setTerminar(boolean terminar) { this.terminar = terminar; }
    public boolean isRobotAberto() { return robotAberto; }
    public void setRobotAberto(boolean robotAberto) { this.robotAberto = robotAberto; }
```

```
public void setDistancia(int distancia ) { this.distancia = distancia; }
public int getDistancia() { return distancia;}

public void setAngulo(int angulo) { this.angulo = angulo; }
public int getAngulo() { return angulo;}

public void setRaio(int raio) { this.raio = raio; }
public int getRaio() { return raio;}

public void setNomeRobot(String nomeRobot) { this.nomeRobot = nomeRobot; }
public String getNomeRobot() { return nomeRobot; }

}
```

### 5.3. Classe “GUI.java”

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class GUI extends JFrame {

    private BaseDados bd;
    private static final long serialVersionUID = 1L;
    private JPanel contentPane;
    private JTextField text_Distancia, text_Angulo, text_Raio, text_Robot;
    private JCheckBox chckbxNewCheckBox;
    private JButton btnFrente, btnEsquerda, btnTras, btnDireita, btnParar;
    private JSpinner spinner;
    private JRadioButton rdbtnNewRadioButton;
    private JTextArea console;
    private MovimentoAleatorio produtor;
    private Servidor consumidor;
    private BufferCircular buffer;
```

```
private void Consola(String s) {
    console.append(s+"\n");
}

public void setBuffer(BufferCircular buffer) {
    this.buffer = buffer;
}

public GUI() {
    setTitle("EV3 Demo");
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

    contentPane = new JPanel();
    contentPane.setBorder(new javax.swing.border.EmptyBorder(5, 5, 5, 5));
    setContentPane(contentPane);
    contentPane.setLayout(new BorderLayout());

    bd = new BaseDados();

    // Fecho seguro da app
    addWindowListener(new WindowAdapter() {
        @Override public void windowClosing(WindowEvent e) {
            if (bd.isRobotAberto()) {
                bd.getRobot().CloseEV3();
                bd.setRobotAberto(false);
            }
            bd.setTerminar(true);
            dispose();
        }
    });

    // GRELHA DO SISTEMA
```

```

JPanel panelTop = new JPanel();
contentPane.add(panelTop);
GridBagLayout gbl_panelTop = new GridBagLayout();
gbl_panelTop.columnWidths = new int[]{0,0,0,0,0,0,0,0,0};
gbl_panelTop.rowHeights = new int[]{0,0,0,0,0, 0, 0, 0, 0, 0, 0,0,0};
gbl_panelTop.columnWeights = new
double[]{1.0,1.0,1.0,1.0,0.0,1.0,0.0,1.0,Double.MIN_VALUE};
gbl_panelTop.rowWeights = new double[]{0.0,0.0,0.0,0.0,0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
0.0,1.0,Double.MIN_VALUE};
panelTop.setLayout(gbl_panelTop);

```

```
// CAMPO --> RAI0
```

```

JLabel lbl_Raio = new JLabel("Raio");
GridBagConstraints gbc_lbl_Raio = new GridBagConstraints();
gbc_lbl_Raio.anchor = GridBagConstraints.EAST;
gbc_lbl_Raio.insets = new Insets(0,0,5,5);
gbc_lbl_Raio.gridx = 0; gbc_lbl_Raio.gridy = 0;
panelTop.add(lbl_Raio, gbc_lbl_Raio);
text_Raio = new JTextField();
text_Raio.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        bd.setRaio(Integer.parseInt(text_Raio.getText()));
    }
});text_Raio.setColumns(5);
GridBagConstraints gbc_textField = new GridBagConstraints();
gbc_textField.insets = new Insets(0,0,5,5);
gbc_textField.fill = GridBagConstraints.HORIZONTAL;
gbc_textField.gridx = 1; gbc_textField.gridy = 0;
panelTop.add(text_Raio, gbc_textField);

```

```
// CAMPO --> ÂNGULO
```

```
JLabel lbl_Angulo = new JLabel("Ângulo");
```

```
GridBagConstraints gbc_lbl_Angulo = new GridBagConstraints();
gbc_lbl_Angulo.anchor = GridBagConstraints.EAST;
gbc_lbl_Angulo.insets = new Insets(0,0,5,5);
gbc_lbl_Angulo.gridx = 2; gbc_lbl_Angulo.gridy = 0;
panelTop.add(lbl_Angulo, gbc_lbl_Angulo);

text_Angulo = new JTextField();
text_Angulo.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        bd.setAngulo(Integer.parseInt(text_Angulo.getText()));
    }
});text_Angulo.setColumns(1);
GridBagConstraints gbc_text_Angulo = new GridBagConstraints();
gbc_text_Angulo.insets = new Insets(0,0,5,5);
gbc_text_Angulo.fill = GridBagConstraints.HORIZONTAL;
gbc_text_Angulo.gridx = 3; gbc_text_Angulo.gridy = 0;
panelTop.add(text_Angulo, gbc_text_Angulo);

// CAMPO --> DISTÂNCIA
JLabel lbl_Distancia = new JLabel("Distancia");
GridBagConstraints gbc_lbl_Distancia = new GridBagConstraints();
gbc_lbl_Distancia.anchor = GridBagConstraints.EAST;
gbc_lbl_Distancia.insets = new Insets(0,0,5,5);
gbc_lbl_Distancia.gridx = 4; gbc_lbl_Distancia.gridy = 0;
panelTop.add(lbl_Distancia, gbc_lbl_Distancia);

text_Distancia = new JTextField();
text_Distancia.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        bd.setDistancia(Integer.parseInt(text_Distancia.getText()));
    }
});
text_Distancia.setColumns(1);
```

```
GridBagConstraints gbc_text_Distancia = new GridBagConstraints();
gbc_text_Distancia.insets = new Insets(0,0,5,5);
gbc_text_Distancia.fill = GridBagConstraints.HORIZONTAL;
gbc_text_Distancia.gridx = 5; gbc_text_Distancia.gridy = 0;
panelTop.add(text_Distancia, gbc_text_Distancia);
```

```
// CAMPO --> NOME DO ROBOT
JLabel lbl_Robot = new JLabel("Robot");
GridBagConstraints gbc_lbl_Robot = new GridBagConstraints();
gbc_lbl_Robot.anchor = GridBagConstraints.EAST;
gbc_lbl_Robot.insets = new Insets(0,0,5,5);
gbc_lbl_Robot.gridx = 6; gbc_lbl_Robot.gridy = 0;
panelTop.add(lbl_Robot, gbc_lbl_Robot);
```

```
text_Robot = new JTextField();
text_Robot.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        String nome = text_Robot.getText();
        bd.setNomeRobot(text_Robot.getText());
        Consola("Nome do robot definido:" + nome);
    }
});text_Robot.setColumns(1);
GridBagConstraints gbc_text_Robot = new GridBagConstraints();
gbc_text_Robot.insets = new Insets(0,0,5,0);
gbc_text_Robot.fill = GridBagConstraints.HORIZONTAL;
gbc_text_Robot.gridx = 7; gbc_text_Robot.gridy = 0;
panelTop.add(text_Robot, gbc_text_Robot);
```

```
// CAIXA --> ESTABELECEER LIGAÇÃO COM O ROBOT --EV10--
chckbxNewCheckBox = new JCheckBox("Ligar");
chckbxNewCheckBox.addActionListener(new ActionListener() {
```



```
public void actionPerformed(ActionEvent e) {
    if (!bd.isRobotAberto()) {
        String nome = bd.getNomeRobot();
        boolean ok = bd.getRobot().OpenEV3(nome);
        bd.setRobotAberto(ok);
        chckbxNewCheckBox.setSelected(ok);
        Consola(ok ? "Robot conectado: " + nome
                : " Falha na ligação: " + nome);

        if (ok && produtor != null && consumidor != null) {
            produtor.desbloquear();
            consumidor.desbloquear();
        }
    } else {
        if (bd.isRobotAberto()) {
            bd.getRobot().CloseEV3();
            bd.setRobotAberto(false);
        }
        Consola("Robot desligado.");

        if (produtor != null && consumidor != null) {
            produtor.bloquear();
            consumidor.bloquear();
        }
    }
}

});

GridBagConstraints gbc_chckbxNewCheckBox = new GridBagConstraints();
gbc_chckbxNewCheckBox.insets = new Insets(0,0,5,5);
gbc_chckbxNewCheckBox.gridx = 6; gbc_chckbxNewCheckBox.gridy = 1;
panelTop.add(chckbxNewCheckBox, gbc_chckbxNewCheckBox);

// BOTÃO --> FRENTE
```

```
btnFrente = new JButton("FRENTE");
btnFrente.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {

        if (!bd.isRobotAberto()) {
            Consola("Necessita de ligar o robot primeiro!");
            return;
        }

        int distancia = bd.getDistancia();
        if (distancia >= 10 && distancia <= 50) {

            buffer.inserirElemento(Comando.retaFrente(distancia));
            buffer.inserirElemento(Comando.parar());

            //bd.getRobot().Reta(distancia);
            //bd.getRobot().Parar(false);

            Consola("Fazer Reta | Distância = " + distancia);
        } else {
            Consola("Distância precisa de estar entre 10 e 50 cm");
        }
    }
});
btnFrente.setBackground(Color.GREEN);
btnFrente.setOpaque(true);
btnFrente.setContentAreaFilled(true);
GridBagConstraints gbc_btnFrente = new GridBagConstraints();
gbc_btnFrente.insets = new Insets(0,0,5,5);
gbc_btnFrente.gridx = 4; gbc_btnFrente.gridy = 2;
panelTop.add(btnFrente, gbc_btnFrente);
```

```
// BOTÃO --> ESQUERDA
```

```
btnEsquerda = new JButton("ESQUERDA");
btnEsquerda.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        if (!bd.isRobotAberto()) {
            Consola("Necessita de ligar o robot primeiro!");
            return;
        }
        int raio = bd.getRaio();
        int angulo = bd.getAngulo();
        if (raio >= 10 && raio <= 30 && angulo >= 20 && angulo <= 90) {

            buffer.inserirElemento(Comando.curvaEsq(raio, angulo));
            buffer.inserirElemento(Comando.parar());

            //bd.getRobot().CurvarEsquerda(raio, angulo);
            //bd.getRobot().Parar(false);

            Consola("Virar à esquerda | Raio = " + raio + "; Ângulo = " + angulo);
        } else {
            Consola("Raio necessita de estar entre 10 e 30 cm | "
                + "Ângulo necessita de estar entre 20 e 90 graus.");
        }
    }
});
btnEsquerda.setBackground(Color.MAGENTA);
btnEsquerda.setOpaque(true);
btnEsquerda.setContentAreaFilled(true);
GridBagConstraints gbc_btnEsquerda = new GridBagConstraints();
gbc_btnEsquerda.insets = new Insets(0,0,5,5);
gbc_btnEsquerda.gridx = 3; gbc_btnEsquerda.gridy = 3;
panelTop.add(btnEsquerda, gbc_btnEsquerda);
```

```
// BOTÃO --> PARAR
```

```
btnParar = new JButton("PARAR");
btnParar.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        if (!bd.isRobotAberto()) {
            Consola("Necessita de ligar o robot primeiro!");
            return;
        }
        try {

            buffer.inserirElemento(Comando.parar());

            //bd.getRobot().Parar(true);

            Consola("A parar o robot.");
        } catch (Exception ex) {
            Consola("Erro detetado ao tentar parar." + ex.getMessage());
        }
    }
});
btnParar.setBackground(Color.RED);
btnParar.setOpaque(true);
btnParar.setContentAreaFilled(true);
GridBagConstraints gbc_btnParar = new GridBagConstraints();
gbc_btnParar.insets = new Insets(0,0,5,5);
gbc_btnParar.gridx = 4; gbc_btnParar.gridy = 3;
panelTop.add(btnParar, gbc_btnParar);
```

// BOTÃO --> DIREITA

```
btnDireita = new JButton("DIREITA");
btnDireita.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        if (!bd.isRobotAberto()) {
            Consola("Necessita de ligar o robot primeiro!");
```

```

        return;
    }
    int raio = bd.getRaio();
    int angulo = bd.getAngulo();
    if (raio >= 10 && raio <= 30 && angulo >= 20 && angulo <= 90) {

        buffer.inserirElemento(Comando.curvaDir(raio, angulo));
        buffer.inserirElemento(Comando.parar());

        //bd.getRobot().CurvarDireita(raio, angulo);
        //bd.getRobot().Parar(false);

        Consola("Virar à direita | Raio = " + raio + "; Ângulo = " + angulo);
    } else {
        Consola("Raio necessita de estar entre 10 e 30 cm | "
            + "Ângulo necessita de estar entre 20 e 90 graus.");
    }
}

});
btnDireita.setBackground(Color.BLUE);
btnDireita.setOpaque(true);
btnDireita.setContentAreaFilled(true);
GridBagConstraints gbc_btnDireita = new GridBagConstraints();
gbc_btnDireita.insets = new Insets(0,0,5,5);
gbc_btnDireita.gridx = 5; gbc_btnDireita.gridy = 3;
panelTop.add(btnDireita, gbc_btnDireita);

//BOTÃO --> TRÁS
btnTras = new JButton("TRÁS");
btnTras.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        if (!bd.isRobotAberto()) {
            Consola("Necessita de ligar o robot primeiro!");

```

```
        return;
    }

    int distancia = bd.getDistancia();
    if (distancia >= 10 && distancia <= 50) {

        buffer.inserirElemento(Comando.retaTras(distancia)); // <<<<< desta forma o
sinal negativo fica na classe comando

        buffer.inserirElemento(Comando.parar());

        //bd.getRobot().Reta(-distancia); // <<<<< nota o sinal negativo
        //bd.getRobot().Parar(false);

        Consola("Fazer Marcha-atrás | Distância = " + distancia);
    } else {
        Consola("Distância precisa de estar entre 10 e 50 cm");
    }
}

});
btnTras.setBackground(Color.PINK);
btnTras.setOpaque(true);
btnTras.setContentAreaFilled(true);
GridBagConstraints gbc_btnTras = new GridBagConstraints();
gbc_btnTras.insets = new Insets(0,0,5,5);
gbc_btnTras.gridx = 4; gbc_btnTras.gridy = 4;
panelTop.add(btnTras, gbc_btnTras);

// TO-DO --> NÚMERO
JLabel lbl_Numero = new JLabel("Número");
GridBagConstraints gbc_lbl_Numero = new GridBagConstraints();
gbc_lbl_Numero.anchor = GridBagConstraints.EAST;
gbc_lbl_Numero.insets = new Insets(0, 0, 5, 5);
```

```
gbc_lbl_Número.gridx = 4;
gbc_lbl_Número.gridy = 5;
panelTop.add(lbl_Número, gbc_lbl_Número);

spinner = new JSpinner();
GridBagConstraints gbc_spinner = new GridBagConstraints();
gbc_spinner.anchor = GridBagConstraints.WEST;
gbc_spinner.insets = new Insets(0, 0, 5, 5);
gbc_spinner.gridx = 5;
gbc_spinner.gridy = 5;
panelTop.add(spinner, gbc_spinner);

//TO-DO --> MOVIMENTOS ALEATÓRIOS
rdbtnNewRadioButton = new JRadioButton("Movimentos Aleatórios");
rdbtnNewRadioButton.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        if (!bd.isRobotAberto()) {
            rdbtnNewRadioButton.setSelected(false);
            Consola("Necessita de ligar o robot primeiro!");
            return;
        }
        if (produtor == null) {
            Consola("Produtor não disponível.");
            rdbtnNewRadioButton.setSelected(false);
            return;
        }
        int n = (int) spinner.getValue();
        produtor.iniciarComandos(n);
        Consola("Pedido de lote aleatório: " + n + " comandos (+ PARAR).");
        rdbtnNewRadioButton.setSelected(false);    // opcional: desmarca
    }
});

GridBagConstraints gbc_rdbtnNewRadioButton = new GridBagConstraints();
gbc_rdbtnNewRadioButton.insets = new Insets(0, 0, 5, 5);
```

```
gbc_rdbtnNewRadioButton.gridx = 6;
gbc_rdbtnNewRadioButton.gridy = 5;
panelTop.add(rdbtnNewRadioButton, gbc_rdbtnNewRadioButton);

// CONSOLA --> HISTÓRICO DE AÇÕES
JLabel lbl_Consola = new JLabel("Consola");
GridBagConstraints gbc_lbl_Consola = new GridBagConstraints();
gbc_lbl_Consola.insets = new Insets(0, 0, 5, 5);
gbc_lbl_Consola.gridx = 2;
gbc_lbl_Consola.gridy = 6;
panelTop.add(lbl_Consola, gbc_lbl_Consola);

console = new JTextArea();
console.setEditable(false);
console.setLineWrap(true);
console.setWrapStyleWord(true);

JScrollPane scroll = new JScrollPane(console);

GridBagConstraints gbc_console = new GridBagConstraints();
gbc_console.gridx = 2;
gbc_console.gridy = 7;
gbc_console.gridwidth = 6;
gbc_console.gridheight = 3;
gbc_console.insets = new Insets(0, 0, 5, 0);
gbc_console.fill = GridBagConstraints.BOTH;
gbc_console.weightx = 1.0; // << faz expandir na horizontal
gbc_console.weighty = 1.0; // << faz expandir na vertical
panelTop.add(scroll, gbc_console);
setPreferredSize(new Dimension(600, 400)); // tamanho desejado da janela
pack(); // calcula layout com base nos preferred sizes
setLocationRelativeTo(null);
setVisible(true);
```



```
        pack();
        setLocationRelativeTo(null);
        setVisible(true);
    }

    public void setTarefas(MovimentoAleatorio produtor, Servidor consumidor) {
        this.produtor = produtor;
        this.consumidor = consumidor;
    }

    // <<< O GETTER TEM QUE FICAR AQUI, FORA DO CONSTRUTOR >>>
    public BaseDados getBD() {
        return bd;
    }
}
```

#### **5.4. Classe “Tarefa.java”**

```
import java.util.concurrent.Semaphore;

public abstract class Tarefa extends Thread {
    private final byte BLOQUEADO = 0, EXECUCAO = 1, DORMIR = 2;

    private final Semaphore sem = new Semaphore(0);
    private volatile byte estado = BLOQUEADO;

    public Tarefa() {}

    public void desbloquear() {
        estado = EXECUCAO;
        sem.release();
    }
}
```

```
public void bloquear() {
    estado = BLOQUEADO;
}

public void run() {
    while (true) {
        switch (estado) {
            case BLOQUEADO:
                try {
                    sem.acquire();
                } catch (InterruptedException ignored) {
                }
            case EXECUCAO:
                execucao();
                if (estado == EXECUCAO)
                    estado = DORMIR;
            case DORMIR:
                dormir();
                if (estado == DORMIR)
                    estado = EXECUCAO;
        }
    }
}

protected abstract void execucao();

protected void dormir() {
    try { Thread.sleep(100);
    } catch (InterruptedException ignored) {} }

}
```

### 5.5. Classe “Comando.java”

```
public class Comando {

    public enum Tipo { RETA_FRENTE, RETA_TRAS, CURVA_ESQ, CURVA_DIR,
    PARAR }

    // Apenas dois parametros pois o movimento mais complexo tem no máximo 2 parametros
    public final Tipo tipo;
    public final int p1; // Parametro para todos os movimentos ( Distancia ou Raio )
    public final int p2; // Parametro para movimentos de rotacao ( Angulo )

    public Comando(Tipo t, int p1, int p2) {
        this.tipo = t;
        this.p1 = p1;
        this.p2 = p2;
    }

    public Comando(Comando comando) {
        this.tipo = comando.tipo;
        this.p1 = comando.p1;
        this.p2 = comando.p2;
    }

    // Fábricas estáticas para maior clareza
    public static Comando retaFrente(int dist) {
        System.out.println("TEST");
        return new Comando(Tipo.RETA_FRENTE, dist, 0);
    }

    public static Comando retaTras(int dist) {
        return new Comando(Tipo.RETA_TRAS, dist, 0);
    }

    public static Comando curvaEsq(int raio, int ang) {
        return new Comando(Tipo.CURVA_ESQ, raio, ang);
    }

    public static Comando curvaDir(int raio, int ang) {
```

```
        return new Comando(Tipo.CURVA_DIR, raio, ang);
    }
    public static Comando parar() {
        return new Comando(Tipo.PARAR, 0, 0);
    }
}
```

### 5.6. Classe “BufferCircular.java”

```
import java.util.concurrent.Semaphore;

public class BufferCircular {
    final int dimensaoBuffer = 16; // Capacidade de armazenamento do robot
    private final Comando[] bufferCircular;
    private int getBuffer; // índice de onde o consumidor vai retirar o próximo comando
    private int putBuffer; // índice onde o produtor vai colocar o próximo comando

    // NOTA: Não esquecer que para cada grupo de movimentos aleatorios é sempre contado
    // mais 1
    //instrução para contar com a instrução parar(FALSE) obrigatorio no final das instrucoes

    // criar semaforos
    private final Semaphore acessoElemento;
    private final Semaphore elementosLivres;
    private final Semaphore elementosOcupados;

    // Helper
    public int ocupados() { return elementosOcupados.availablePermits(); }

    public BufferCircular() {
        bufferCircular = new Comando[dimensaoBuffer];
        getBuffer = 0;
        putBuffer = 0;

        elementosLivres = new Semaphore(dimensaoBuffer);
        elementosOcupados = new Semaphore(0);
    }
}
```

```
        acessoElemento = new Semaphore(1);
    }

// _____PRODUTOR_____

public void inserirElemento(Comando comando){
    try {

        if (elementosLivres.availablePermits() == 0) {
            System.out.println("[BUFFER] Buffer cheio (produtor vai aguardar espaço livre)");
        }

        elementosLivres.acquire();
        acessoElemento.acquire();

        bufferCircular[putBuffer]= comando;
        putBuffer = (putBuffer + 1) % dimensaoBuffer;

    } catch (InterruptedException e) {
        e.printStackTrace();
    } finally {
        acessoElemento.release();
        // primeiro liberta um "ocupado"
        elementosOcupados.release();
        System.out.println("[BUFFER] Inserido (" + comando.tipo + "). Ocupados: "
            + elementosOcupados.availablePermits() + "/" + dimensaoBuffer);
    }
}

// _____CONSUMIDOR_____

public Comando removerElemento() {
    Comando comando= null;
    try {
        if (elementosOcupados.availablePermits() == 0) {
            System.out.println("[BUFFER] Vazio (consumidor vai aguardar)");
        }
    }
```

```
        elementosOcupados.acquire();
        acessoElemento.acquire();

        comando = bufferCircular[getBuffer];
        getBuffer = (getBuffer + 1) % dimensaoBuffer;

    } catch (InterruptedException e) {
        e.printStackTrace();
    } finally {
        acessoElemento.release();
        elementosLivres.release();
        System.out.println("[BUFFER] Removido (" + comando.tipo + "). Ocupados: "
            + elementosOcupados.availablePermits() + "/" + dimensaoBuffer);

        if (elementosOcupados.availablePermits() == 0) {
            System.out.println("[BUFFER] (Vazio após consumir tudo)");
        }
    }
    return comando;
}

}
```

### 5.7. Classe “Servidor.java”

```
public class Servidor extends Tarefa {

    private final BufferCircular buffer;
    private final RobotLegoEV3 robot;
    private final BaseDados bd;

    /*
     * Classe Consumidora
     * Executa os comandos no Robot
     */

    public Servidor(BufferCircular buffer, RobotLegoEV3 robot, BaseDados bd) {
```

```
        super();
        this.buffer = buffer;
        this.robot = robot;
        this.bd = bd;    }

// Métodos separados (bons para depuração e modularidade)
public void Reta(int distancia) {
    robot.Reta(distancia);
}

public void CurvarDireita(int raio, int angulo) {
    robot.CurvarDireita(raio, angulo);
}

public void CurvarEsquerda(int raio, int angulo) {
    robot.CurvarEsquerda(raio, angulo);
}

public void Parar() {
    robot.Parar(false);
}

// Método principal do consumidor (executa o comando retirado do buffer)
public void executar(Comando comando) {

    switch (comando.tipo) {
        case RETA_FRENTE:
            Reta(comando.p1);
            break;
        case RETA_TRAS:
            Reta(-comando.p1);
            break;
        case CURVA_DIR:
            CurvarDireita(comando.p1, comando.p2);
            break;
```

```

        case CURVA_ESQ:
            CurvarEsquerda(comando.p1, comando.p2);
            break;
        case PARAR:
            Parar();
            break;
    }
}

```

@Override

```

protected void execucao() {
    if (!bd.isRobotAberto()) { bloquear(); return; }

```

```

    // 1) aguarda 1 comando (bloqueante)

```

```

    Comando c = buffer.removerElemento();
    executar(c);

```

```

    // 2) drena os restantes que já estejam disponíveis (sem dormir)

```

```

    while (buffer.ocupados() > 0) {
        c = buffer.removerElemento();
        executar(c);
    }
}

```

@Override

```

protected void dormir() {
    //try { Thread.sleep(5000); } catch (InterruptedException ignored) {}
}
}

```

## 5.8. Classe “MovimentoAleatorio.java”

```

import java.util.Random;

```

```

public class MovimentoAleatorio extends Tarefa{

```



```
private final BufferCircular buffer;
private final BaseDados bd;
private final Random rnd = new Random();

private volatile int pendentes = 0; // Tamanho do nr de comandos pedidos na GUI

    public MovimentoAleatorio(BufferCircular buffer, BaseDados bd) {
super();
this.buffer = buffer;
this.bd = bd;
    }

    public void iniciarComandos(int n) {
pendentes = Math.max(1, n);
desbloquear();          // acorda a tarefa para produzir o lote
    }

    private Comando comandoAleatorio() {
        int tipo = rnd.nextInt(3); // 0=RETA_FRENTE, 1=CURVA_ESQ,
2=CURVA_DIR

        // Não é permitido os valores de distancia, raio e angulo serem menores que 10,
10, 30, respetivamente

        switch (tipo) {
            case 0: { // RETA_FRENTE
                int dist = 10 + rnd.nextInt(41); // 10..50 cm (ajusta à tua GUI)
                return Comando.retaFrente(dist);
            }
            case 1: { // CURVA_ESQ
                int raio = 10 + rnd.nextInt(21); // 10..30
                int ang = 30 + rnd.nextInt(61); // 30..90
                return Comando.curvaEsq(raio, ang);
            }
            default: { // CURVA_DIR
```

```

        int raio = 10 + rnd.nextInt(21);
        int ang  = 30 + rnd.nextInt(61);
        return Comando.curvaDir(raio, ang);
    }
}

// ----- Máquina de estados herdada de Tarefa -----
@Override
protected void execucao() {
    // só produz se o robot estiver ligado e houve pedido
    if (!bd.isRobotAberto() || pendentes <= 0) { bloquear(); return; }

    // produzir exatamente 'pendentes' comandos
    for (int i = 0; i < pendentes; i++) {
        buffer.inserirElemento(comandoAleatorio());
        try { Thread.sleep(50); } catch (InterruptedException ignored) {}
    }
    buffer.inserirElemento(Comando.parar()); // fecha o lote

    pendentes = 0; // esgota o pedido
    bloquear();    // volta a BLOQUEADO até novo clique da GUI
}

@Override
protected void dormir() {
    // podes reduzir a cadência quando "desativado"
    try { Thread.sleep(200); } catch (InterruptedException ignored) {}
}
}

```

### 5.9. Classe “RobotLegoEV3SIM.java”

```
import java.util.function.Consumer;
```

```
public class RobotLegoEV3SIM extends RobotLegoEV3 {

    private final Consumer<String> log;
    private volatile boolean ligado = false;

    // atraso “realista” por cm / por grau (ajusta como quiseses)
    private static final int MS_POR_CM = 40; // ex.: 20–60
    private static final int MS_POR_GRAU = 15; // ex.: 10–25

    public RobotLegoEV3SIM() {
        this(System.out::println);
    }

    public RobotLegoEV3SIM(Consumer<String> logger) {
        this.log = (logger != null) ? logger : System.out::println;
    }

    @Override
    public boolean OpenEV3(String nome) {
        ligado = true;
        log.accept("[SIM] OpenEV3(" + nome + ") -> true");
        return true;
    }

    @Override
    public void CloseEV3() {
        ligado = false;
        log.accept("[SIM] CloseEV3()");
    }

    @Override
    public void Reta(int distancia) {
        if (!ligado) {
            log.accept("[SIM] O comando 'Reta' foi ignorado (robot OFF)");
            return;
        }
    }
}
```

```
    }
    log.accept("[SIM] Reta(Frente, " + distancia + " cm");
    dormir(Math.abs(distancia) * MS_POR_CM);
}

public void Tras(int distancia) {
    if (!ligado) {
        log.accept("[SIM] O comando 'Tras' foi ignorado (robot OFF)");
        return;
    }
    log.accept("[SIM] Reta(Trás, " + distancia + " cm");
    dormir(Math.abs(distancia) * MS_POR_CM);
}

@Override
public void CurvarEsquerda(int raio, int angulo) {
    if (!ligado) { log.accept("[SIM] O comando 'CurvarEsquerda' foi ignorado (robot OFF)");
return; }
    log.accept("[SIM] CurvarEsquerda(raio=" + raio + ", ang=" + angulo + ")");
    dormir((int)(raio * MS_POR_CM * 0.5 + Math.abs(angulo) * MS_POR_GRAU));
}

@Override
public void CurvarDireita(int raio, int angulo) {
    if (!ligado) { log.accept("[SIM] O comando 'CurvarDireita' foi ignorado (robot OFF)");
return; }
    log.accept("[SIM] CurvarDireita(raio=" + raio + ", ang=" + angulo + ")");
    dormir((int)(raio * MS_POR_CM * 0.5 + Math.abs(angulo) * MS_POR_GRAU));
}

@Override
public void Parar(boolean imediato) {
    if (!ligado) { log.accept("[SIM] O comando 'Parar' foi ignorado (robot OFF)"); return; }
    log.accept("[SIM] Parar(imediato=" + imediato + ")");
    if (!imediato) dormir(1000);
}
```

```
}
```

```
private void dormir(int ms) {  
    try { Thread.sleep(Math.max(0, ms)); }  
    catch (InterruptedException ignored) {}  
}  
}
```